

# Cloud Service Models

**Task-1 List three popular apps you use (like Instagram, Zomato, or Spotify) and for each, identify whether it is most similar to IaaS, PaaS, or SaaS. Briefly explain your reasoning for each choice.**

| App       | Model | Reason                                                                                                                                   |
|-----------|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| Instagram | SaaS  | Users access a ready-to-use application for sharing photos, videos, and messages without managing servers or software.                   |
| Spotify   | SaaS  | Users directly use Spotify's finished music-streaming application; the underlying infrastructure is managed by Spotify.                  |
| Zomato    | SaaS  | Customers use a complete application to search restaurants, order food, and track deliveries without managing the underlying technology. |

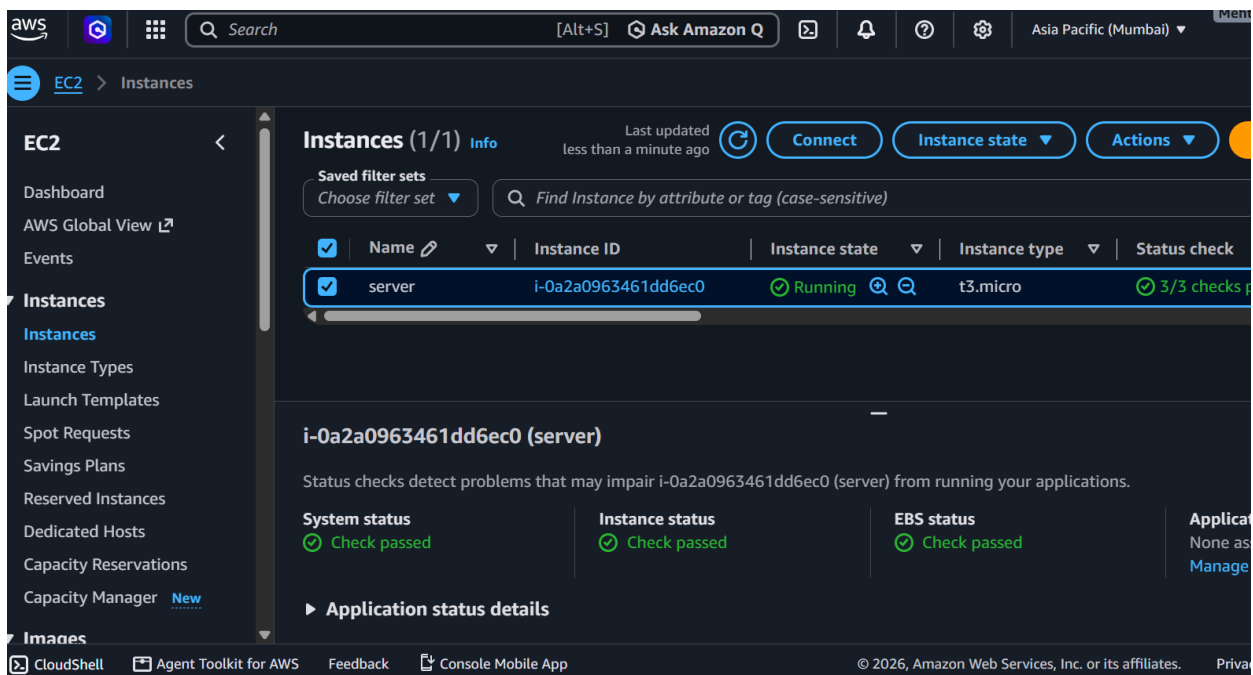
These apps are technically built using infrastructure and platform services internally, but from the end user's perspective, they are SaaS.

**Task-2 Create a table comparing IaaS, PaaS, and SaaS with at least three points for each model, using real-world examples such as AWS EC2, Google App Engine, and Office 365.**

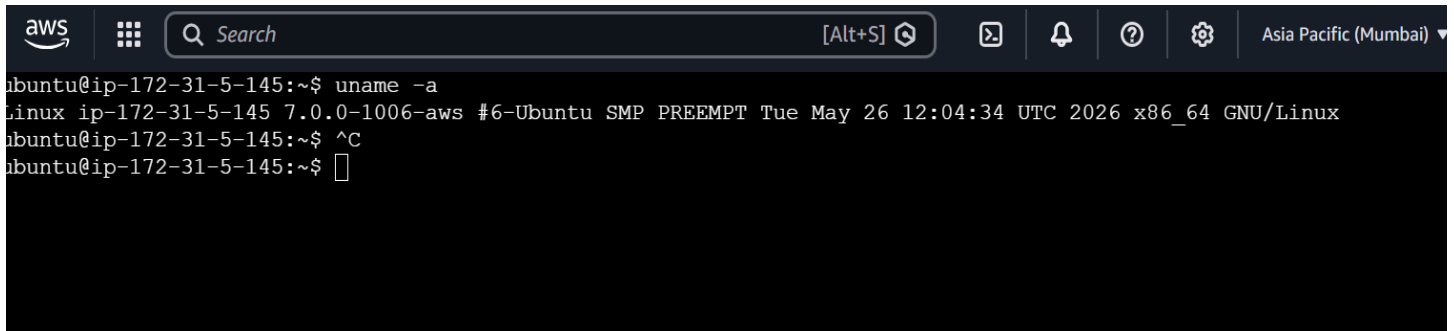
| Point                 | IaaS                        | PaaS                         | SaaS                                  |
|-----------------------|-----------------------------|------------------------------|---------------------------------------|
| Meaning               | Infrastructure as a Service | Platform as a Service        | Software as a Service                 |
| What the user manages | OS, applications, and data  | Mainly applications and data | Mostly just application settings/data |

|              |                                                            |                                                     |                                                  |
|--------------|------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Point        | IaaS                                                       | PaaS                                                | SaaS                                             |
| Main benefit | Provides flexible virtual servers, storage, and networking | Makes application development and deployment easier | Provides ready-to-use software over the internet |
| Example      | AWS EC2                                                    | Google App Engine                                   | Microsoft 365 (Office 365)                       |
| Typical user | System administrators/developers                           | Application developers                              | End users/business users                         |

**Task-3 Sign up for a free AWS account (or use an existing one), navigate to the EC2 dashboard, and launch a new t3.micro EC2 instance running Ubuntu 22.04. Take a screenshot of your running instance in the AWS console. Use the 'Launch Instance' button and select the free tier eligible Ubuntu AMI.**



**Task-4** After your EC2 instance is running, connect to it using SSH from your local terminal or an online SSH client. Run the command 'uname -a' and note the output. You may need to use the .pem key file downloaded during instance setup.



```
aws | [Search] [Alt+S] | [Icons] | Asia Pacific (Mumbai) ▼
ubuntu@ip-172-31-5-145:~$ uname -a
Linux ip-172-31-5-145 7.0.0-1006-aws #6-Ubuntu SMP PREEMPT Tue May 26 12:04:34 UTC 2026 x86_64 GNU/Linux
ubuntu@ip-172-31-5-145:~$ ^C
ubuntu@ip-172-31-5-145:~$
```

**Task-5** Suppose you want to host a simple static website for your college fest using your EC2 instance. List the basic steps you would follow after launching the instance to make your site accessible to everyone. Write 3-5 steps in order.

1. Connect to the Ubuntu EC2 instance using SSH and update the system packages.
2. Install and start the Apache web server using `sudo apt install apache2 -y`.
3. Create the college-fest index.html file inside `/var/www/html/`.
4. Configure the EC2 Security Group to allow incoming HTTP traffic on port 80.
5. Copy the EC2 instance's Public IPv4 address and open it in a browser to access the website.



## Apache2 Default Page

# Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

### Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/  
|-- apache2.conf  
|   |-- ports.conf  
|-- mods-enabled
```